

Week-7 (DAY-32, Day-33) Hands-On – Keerthivasan R

Problem: Contact Management System using Layered Architecture, Repository Pattern & Dependency Injection

Scenario:

ABC Infotech wants to build a **Contact Management System** where each contact belongs to a **Company** and optionally to a **Department** within that company.

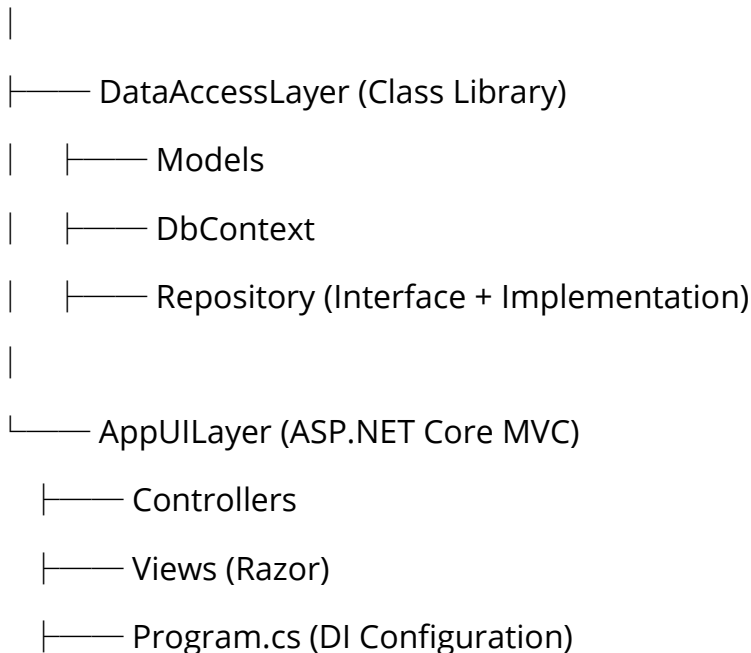
The system should:

- Maintain **company and department master data**
- Allow users to perform **CRUD only on ContactInfo**
- Establish proper **relationships between entities**
- Follow **Layered Architecture + Repository Pattern + Dependency Injection**

Architecture Overview

Solution Structure

Solution: ContactManagementSystem



Requirements:

1.Data Access Layer/Service Layer (Class Library).

- ◆ Model Class → ContactInfo (Main Entity)

Name	Type	Modifier
ContactId	int	public
FirstName	string	public
LastName	string	public
EmailId	string	public
MobileNo	long	public
Designation	string	public
CompanyId	int	public (FK)
DepartmentId	int	public (FK)

- ◆ Model 2 → Company

Name	Type
CompanyId	int
CompanyName	string

- ◆ Model 3 → Department

Name	Type
DepartmentId	int
DepartmentName	string

- ◆ **DbContext** → **AppDbContext**

```
public DbSet<ContactInfo> Contacts { get; set; }  
public DbSet<Company> Companies { get; set; }  
public DbSet<Department> Departments { get; set; }
```

- ◆ **Relationships:** (Create relationship by overriding OnModelCreating() of DbContext using fluent api)
 - One Company → Many Contacts
 - One Department → Many Contact
- ◆ **Configure Connection** (Configure connection by overriding OnConfiguring () of DbContext)
- ◆ **Database Setup**
 - Use EF Core Code-First
 - Add migrations and update database
 - Seed initial data (optional but recommended):
- ◆ **Repository Pattern**

Interface → IContactRepository

 - List<ContactInfo> GetAllContacts()
 - ContactInfo GetContactById(int id)
 - void AddContact(ContactInfo contact)
 - void UpdateContact(ContactInfo contact)
 - void DeleteContact(int id)

Implementation → ContactRepository

2. App UI Layer

- ◆ **Controller → ContactController**
 - Inject IContactRepository via DI
- ◆ Action Methods

Action	Description
ShowContacts()	Display all contacts with Company & Department names
GetContactById(int id)	Show full contact details
AddContact()	Show form with dropdowns
[HttpPost] AddContact(ContactInfo contact)	Save and redirect
EditContact(int id)	Load data into form
[HttpPost] EditContact(ContactInfo contact)	Update
DeleteContact(int id)	Delete

◆ Views (Important Enhancement)

- In **Add/Edit Contact View**:
 - Provide dropdowns:
 - Company
 - Department
- Pass dropdown data using:
 - ViewBag OR ViewData

◆ Routing

- Use **Attribute-based routing**

◆ Dependency Injection

```
builder.Services.AddScoped<IContactRepository, ContactRepository>();
```

Technical Constraints

1. MUST use:

- EF Core Code-First
- Relationships (FK + Navigation Properties)
- Repository Pattern
- Dependency Injection
- Attribute Routing
- Razor Views

2. CRUD ONLY on:

- ☒ ContactInfo
- ☒ Company & Department (read-only dropdowns)

3. DO NOT:

- Write DB logic in Controller ☒
- Use static lists ☒

Expectations

1. Correct implementation of relationships
2. Dropdown binding for foreign keys
3. Proper use of Include()
4. Clean layered architecture
5. Fully working CRUD on ContactInfo

Learning Outcome

1.EF Core:

- Migrations
- Code-First relationships

2.DI + Repository Pattern

3. Industry Readiness

1. Build real-world **multi-entity systems**
2. Understand how UI interacts with relational data

CODE:

 DataAccessLayer/ Models/ [Company.cs](#)

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace DataAccessLayer.Models
{
    public class Company
    {
        public int CompanyId { get; set; }
        public string CompanyName { get; set; }

        public List<ContactInfo> Contacts { get; set; }
    }
}
```

 DataAccessLayer/ Models/ [ContactInfo.cs](#)

```
using System;
using System.Collections.Generic;
using System.ComponentModel.DataAnnotations;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using static Microsoft.EntityFrameworkCore.DbLoggerCategory.Database;

namespace DataAccessLayer.Models
{

```

```

public class ContactInfo
{
    [Key]
    public int ContactId { get; set; }

    [Required(ErrorMessage = "First Name is required")]
    public string FirstName { get; set; }

    [Required(ErrorMessage = "Last Name is required")]
    public string LastName { get; set; }

    [Required(ErrorMessage = "Email is required")]
    [EmailAddress(ErrorMessage = "Invalid Email")]
    public string EmailId { get; set; }

    [Required(ErrorMessage = "Mobile number is required")]
    [Range(1000000000, 9999999999, ErrorMessage = "Enter valid 10-digit mobile number")]
    public long MobileNo { get; set; }

    [Required(ErrorMessage = "Designation is required")]
    public string Designation { get; set; }

    [Required(ErrorMessage = "Select Company")]
    public int CompanyId { get; set; }

    [Required(ErrorMessage = "Select Department")]
    public int DepartmentId { get; set; }

    public Company? Company { get; set; }
    public Department? Department { get; set; }
}
}

```

DataAccessLayer/ Models/ Department.cs

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace DataAccessLayer.Models
{
    public class Department
    {
        public int DepartmentId { get; set; }
        public string DepartmentName { get; set; }

        public List<ContactInfo> Contacts { get; set; }
    }
}

```

DataAccessLayer/ Repository/ IContactRepository.cs

```

using DataAccessLayer.Models;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace DataAccessLayer.Repository
{
    public interface IContactRepository
    {
        List<ContactInfo> GetAllContacts();
        ContactInfo GetContactById(int id);
        void AddContact(ContactInfo contact);
    }
}

```

```

        void UpdateContact(ContactInfo contact);
        void DeleteContact(int id);
        List<Company> GetCompanies();
        List<Department> GetDepartments();
    }
}

```

DataAccessLayer/ Repository/ ContactRepository.cs

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using DataAccessLayer.Models;
using Microsoft.EntityFrameworkCore;

namespace DataAccessLayer.Repository
{
    public class ContactRepository : IContactRepository
    {
        private readonly AppDbContext _context;

        public ContactRepository(AppDbContext context)
        {
            _context = context;
        }

        public List<Company> GetCompanies()
        {
            return _context.Companies.ToList();
        }

        public List<Department> GetDepartments()
        {
            return _context.Departments.ToList();
        }

        public List<ContactInfo> GetAllContacts()
        {
            return _context.Contacts
                .Include(c => c.Company)
                .Include(c => c.Department)
                .ToList();
        }

        public ContactInfo GetContactById(int id)
        {
            return _context.Contacts
                .Include(c => c.Company)
                .Include(c => c.Department)
                .FirstOrDefault(c => c.ContactId == id);
        }

        public void AddContact(ContactInfo contact)
        {
            _context.Contacts.Add(contact);
            _context.SaveChanges();
        }

        public void UpdateContact(ContactInfo contact)
        {
            _context.Contacts.Update(contact);
            _context.SaveChanges();
        }

        public void DeleteContact(int id)
        {

```

```

        var contact = _context.Contacts.Find(id);
        if (contact != null)
        {
            _context.Contacts.Remove(contact);
            _context.SaveChanges();
        }
    }
}

```

DataAccessLayer/ Data/ AppDbContext.cs

```

using Microsoft.EntityFrameworkCore;
using DataAccessLayer.Models;

public class AppDbContext : DbContext
{
    public AppDbContext(DbContextOptions<AppDbContext> options)
        : base(options)
    {
    }

    public DbSet<ContactInfo> Contacts { get; set; }
    public DbSet<Company> Companies { get; set; }
    public DbSet<Department> Departments { get; set; }

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        modelBuilder.Entity<ContactInfo>()
            .HasOne(c => c.Company)
            .WithMany(c => c.Contacts)
            .HasForeignKey(c => c.CompanyId);

        modelBuilder.Entity<ContactInfo>()
            .HasOne(c => c.Department)
            .WithMany(d => d.Contacts)
            .HasForeignKey(c => c.DepartmentId);

        modelBuilder.Entity<Company>().HasData(
            new Company { CompanyId = 1, CompanyName = "CTS" },
            new Company { CompanyId = 2, CompanyName = "TCS" }
        );

        modelBuilder.Entity<Department>().HasData(
            new Department { DepartmentId = 1, DepartmentName = "HR" },
            new Department { DepartmentId = 2, DepartmentName = "IT" }
        );
    }
}

```

AppUILayer/ Models/ ErrorViewModel.cs

```

namespace AppUILayer.Models
{
    public class ErrorViewModel
    {
        public string? RequestId { get; set; }

        public bool ShowRequestId => !string.IsNullOrEmpty(RequestId);
    }
}

```

AppUILayer/ Controllers/ ContactController.cs

```

using DataAccessLayer.Models;
using DataAccessLayer.Repository;
using Microsoft.AspNetCore.Mvc;

```

```

using Microsoft.AspNetCore.Mvc.Rendering;
using Microsoft.EntityFrameworkCore;

namespace AppUILayer.Controllers
{

    public class ContactController : Controller
    {
        private readonly IContactRepository _repo;

        public ContactController(IContactRepository repo)
        {
            _repo = repo;
        }

        public IActionResult ShowContacts()
        {
            var contacts = _repo.GetAllContacts();
            return View(contacts);
        }

        public IActionResult AddContact()
        {
            LoadDropdowns();
            return View();
        }

        [HttpPost]
        public IActionResult AddContact(ContactInfo contact)
        {
            if (!ModelState.IsValid)
            {
                LoadDropdowns();
                return View(contact);
            }

            _repo.AddContact(contact);
            return RedirectToAction("ShowContacts");
        }

        private void LoadDropdowns()
        {
            ViewBag.Companies = new SelectList(_repo.GetCompanies(), "CompanyId",
"CompanyName");
            ViewBag.Departments = new SelectList(_repo.GetDepartments(), "DepartmentId",
"DepartmentName");
        }

        public IActionResult EditContact(int id)
        {
            var contact = _repo.GetContactById(id);
            LoadDropdowns();
            return View(contact);
        }

        [HttpPost]
        public IActionResult EditContact(ContactInfo contact)
        {
            _repo.UpdateContact(contact);
            return RedirectToAction("ShowContacts");
        }

        public IActionResult DeleteContact(int id)
        {
            _repo.DeleteContact(id);
            return RedirectToAction("ShowContacts");
        }
    }
}

```

AppUILayer/ Views/ Contact/ AddContact.cshtml

@model DataAccessLayer.Models.ContactInfo

<h2>Add Contact</h2>

<form asp-action="AddContact" method="post">

```
<input asp-for="FirstName" class="form-control" placeholder="First Name" />
<span asp-validation-for="FirstName" class="text-danger"></span>
<input asp-for="LastName" class="form-control mt-2" placeholder="Last Name" />
<span asp-validation-for="LastName" class="text-danger"></span>
<input asp-for="EmailId" class="form-control mt-2" placeholder="Email" />
<span asp-validation-for="EmailId" class="text-danger"></span>
<input asp-for="MobileNo" class="form-control mt-2" placeholder="Mobile" />
<span asp-validation-for="MobileNo" class="text-danger"></span>
<input asp-for="Designation" class="form-control mt-2" placeholder="Designation" />
<span asp-validation-for="Designation" class="text-danger"></span>

<select asp-for="CompanyId" asp-items="ViewBag.Companies" class="form-control mt-2">
    <option value="">Select Company</option>
</select>
<span asp-validation-for="CompanyId" class="text-danger"></span>

<select asp-for="DepartmentId" asp-items="ViewBag.Departments" class="form-control mt-2">
    <option value="">Select Department</option>
</select>
<span asp-validation-for="DepartmentId" class="text-danger"></span>

<button type="submit" class="btn btn-success mt-3">Save</button>
```

</form>

<a asp-action="ShowContacts" class="btn-dark">Back to List

```
@section Scripts {
    <partial name="_ValidationScriptsPartial" />
}
```

AppUILayer/ Views/ Contact/ EditContact.cshtml

@model DataAccessLayer.Models.ContactInfo

<h2>Edit Contact</h2>

<form asp-action="EditContact" method="post">

```
<input type="hidden" asp-for="ContactId" />

<input asp-for="FirstName" class="form-control" placeholder="First Name" />
<input asp-for="LastName" class="form-control mt-2" placeholder="Last Name" />
<input asp-for="EmailId" class="form-control mt-2" placeholder="Email" />
<input asp-for="MobileNo" class="form-control mt-2" placeholder="Mobile" />
<input asp-for="Designation" class="form-control mt-2" placeholder="Designation" />

<select asp-for="CompanyId" asp-items="ViewBag.Companies" class="form-control mt-2">
    <option value="">Select Company</option>
</select>

<select asp-for="DepartmentId" asp-items="ViewBag.Departments" class="form-control mt-2">
    <option value="">Select Department</option>
</select>

<button type="submit" class="btn btn-success mt-3">Update</button>
```

</form>

<a asp-action="ShowContacts" class="btn-dark">Back to List

AppUILayer/ Views/ Contact/ ShowContacts.cshtml

```
@model List<DataAccessLayer.Models.ContactInfo>
```

```
<h2>Contacts</h2>
```

```
<a asp-action="AddContact" class="btn btn-primary">Add New Contact</a>
```

```
<table class="table table-bordered mt-3">
```

```
  <thead>
```

```
    <tr>
```

```
      <th>Name</th>
```

```
      <th>Email</th>
```

```
      <th>Company</th>
```

```
      <th>Department</th>
```

```
      <th>Actions</th>
```

```
    </tr>
```

```
  </thead>
```

```
  <tbody>
```

```
    @foreach (var item in Model)
```

```
    {
```

```
      <tr>
```

```
        <td>@item.FirstName @item.LastName</td>
```

```
        <td>@item.EmailId</td>
```

```
        <td>@item.Company?.CompanyName</td>
```

```
        <td>@item.Department?.DepartmentName</td>
```

```
        <td>
```

```
          <a asp-action="EditContact" asp-route-id="@item.ContactId" class="btn btn-  
outline-secondary">Edit</a> |
```

```
          <a asp-action="DeleteContact" asp-route-id="@item.ContactId" class="btn btn-  
danger">Delete</a>
```

```
        </td>
```

```
      </tr>
```

```
    }
```

```
  </tbody>
```

```
</table>
```

AppUILayer/ Program.cs

```
using DataAccessLayer.Repository;  
using Microsoft.EntityFrameworkCore;
```

```
namespace AppUILayer
```

```
{
```

```
  public class Program
```

```
  {
```

```
    public static void Main(string[] args)
```

```
    {
```

```
      var builder = WebApplication.CreateBuilder(args);
```

```
      // Add services to the container.
```

```
      builder.Services.AddControllersWithViews();
```

```
      builder.Services.AddDbContext<AppDbContext>(options =>
```

```
options.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection")));
```

```
      builder.Services.AddScoped<IContactRepository, ContactRepository>();
```

```
      var app = builder.Build();
```

```
      // Configure the HTTP request pipeline.
```

```
      if (!app.Environment.IsDevelopment())
```

```
      {
```

```
        app.UseExceptionHandler("/Home/Error");
```

```
        // The default HSTS value is 30 days. You may want to change this for production  
scenarios, see https://aka.ms/aspnetcore-hsts.
```

```
        app.UseHsts();
```

```
      }
```

```
app.UseHttpsRedirection();
app.UseStaticFiles();

app.UseRouting();

app.UseAuthorization();

app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Contact}/{action=ShowContacts}/{id?}");

app.Run();
}
```

OUTPUT:

AppUILayer Home Privacy

Contacts

Add New Contact

Name	Email	Company	Department	Actions
keerhivasan c	c@gmail.com	CTS	IT	EditDelete

AppUILayer Home Privacy

Add Contact

First Name

Last Name

Email

Mobile

Designation

Select Company

Select Department

Save

Back to List

	ContactId	FirstName	LastName	EmailId	MobileNo	Designation	CompanyId	DepartmentId
▶	1	keerhivasan	c	c@gmail.com	9025877719	a	1	2